Unit –III

Model Interpretability , Post-Hoc Interpretability and Explanations

OUTLINE

1 Model Interpretability: Advances in Interpretable Machine Learning

- Interpretable vs. Explainable Algorithms
- Ensemble-Based Techniques
- Decision Tree-Based Techniques
- Rule-Based Techniques
- Scoring System

2 Post-Hoc Interpretability and Explanations

- Visual Explanation
- Feature Importance
- Example-Based Explanations

Interpretable vs. Explainable Algorithms

Same prediction (Loan Approved) • Different way of understanding the decision

1) INTERPRETABLE MODEL (Transparent by Design)

The decision logic is built into the model.
Humans can directly follow and verify it.

Example: Decision Tree for Loan Approval



Applicant Profile

- Income: ₹6.0 Lakh
- Credit Score: 720
- Debt Ratio: 32%
- Previous Default: No
- Employment: 4 years

Decision Path Followed:
Income > ₹5 Lakh → Credit Score > 700 → Debt Ratio < 40% → **APPROVE**

Key Advantage
The reasoning is part of the model.
Easier to validate, audit, and trust.

Prediction

Loan Approved

2) EXPLAINABLE BLACK-BOX MODEL (Post-hoc Explanation)

The model is complex (e.g., XGBoost / Neural Network).
We use a separate method to explain its prediction.

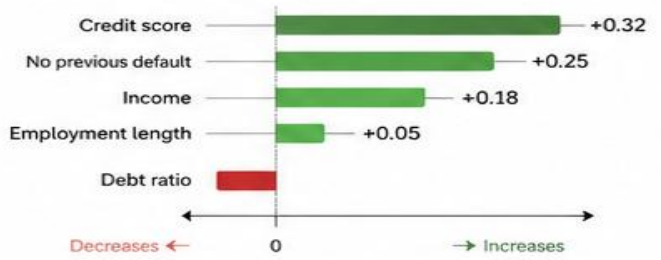
Example: XGBoost Model + SHAP Explanation

XGBoost Model (Black Box) → Prediction: Loan Approved → SHAP → Explanation

Applicant Profile (Same)

- Income: ₹6.0 Lakh
- Credit Score: 720
- Debt Ratio: 32%
- Previous Default: No
- Employment: 4 years

SHAP Feature Contributions



Human Interpretation
The applicant is approved mainly because of good credit score, no previous default and sufficient income.
(Debt ratio has a negative impact.)

Key Limitation
Explanation is an approximation of the black-box model.
May not be 100% faithful; harder to debug.

Key Takeaway
Interpretable models are understandable by design.
Explainable models need an extra step to interpret predictions.

Why it Matters (High-Stakes Decisions)

Credit Scoring · Criminal Justice · Healthcare · Insurance · Compliance & Audit

Interpretable vs. Explainable Algorithms

INTERPRETABLE ALGORITHM

- Understandable by design
- Model structure directly reveals how decisions are made
- Examples: Linear Regression, Decision Tree, Scoring Systems
- Usually one decision model
- Easier to inspect and troubleshoot
- Better suited to high-stakes decisions
- Human can trace the decision pathway

EXPLAINABLE ALGORITHM

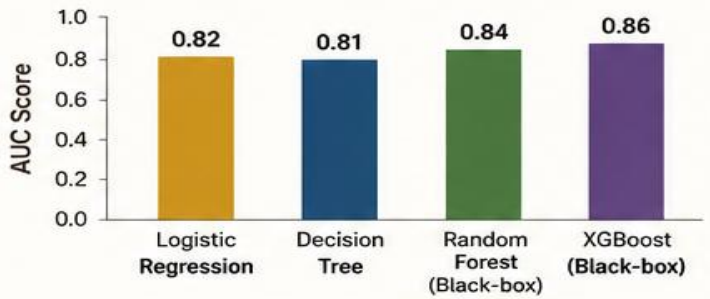
- Explained after training
- A separate technique explains a complex black-box model
- Examples: SHAP, LIME, Saliency Maps
- Two models: black-box + explanation model
- Explanation may be an approximation
- Can create trust and fidelity concerns
- Human sees an interpretation of the pathway

Why Prefer Interpretable Models?

1 Accuracy–Interpretability Trade-off is a Myth

High accuracy does not always require a black-box model.
Interpretable models can achieve comparable accuracy.

Example: Credit Risk Prediction

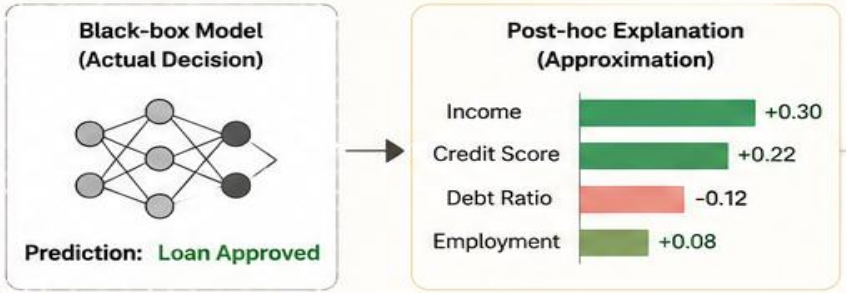


Example

Interpretable models help detect data issues (e.g., wrong encoding, outliers) and improve them iteratively.

2 Post-Hoc Explanations May Not Be Fully Faithful

An explanation is an approximation of the black-box behavior.
It may not exactly represent the model's true decision logic.



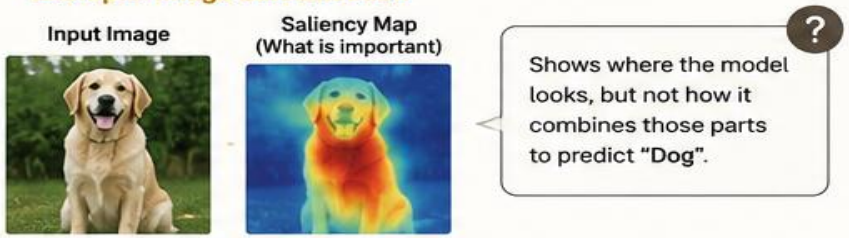
Example

If explanation is 90% accurate, at least 10% of cases may be incorrect—risking wrong approvals or rejections.

3 May Not Reveal the Decision Mechanism

Some methods show which inputs matter, but not how the model combines them to make a prediction.

Example: Image Classification



4 Important Context May Be Missing

Models rely only on available features; real-world decisions may need additional context or domain knowledge.

Example: Bail / Parole Decision

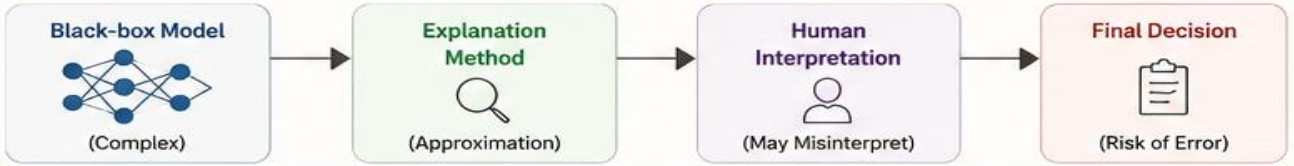


Example

Lack of transparency makes it hard to include such critical context in decisions.

5 Complex Explanations Can Increase Human Error

A complicated black-box + explanation pipeline is harder to understand and follow, increasing the chance of misinterpretation.



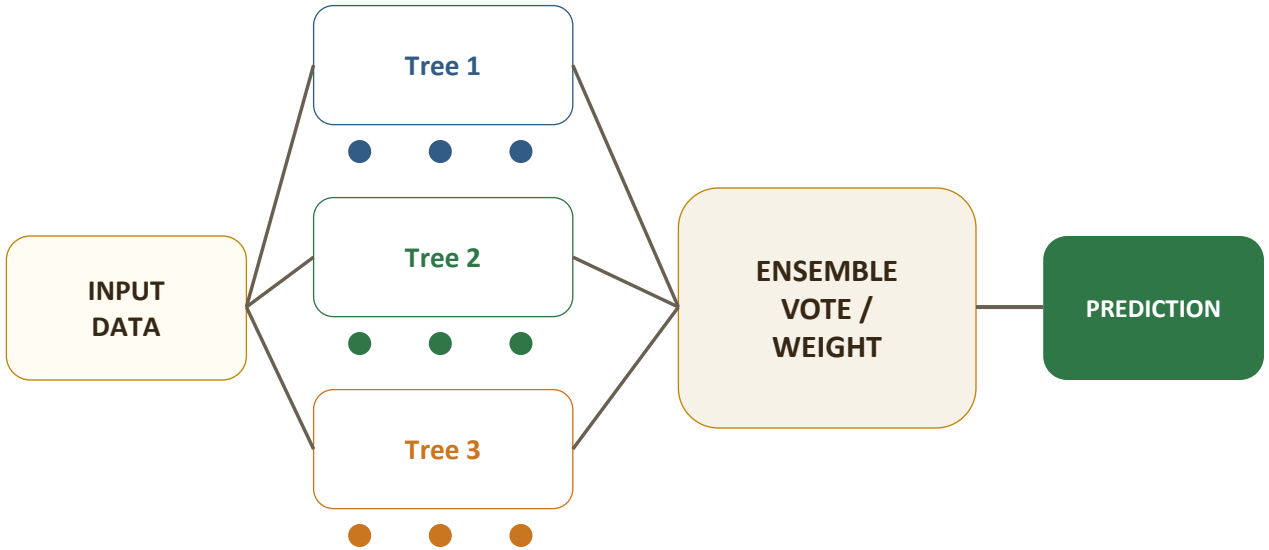
Example

In healthcare, a complex explanation may be misunderstood by clinicians, leading to incorrect treatment decisions.

Ensemble-Based Techniques

Combining multiple models to improve prediction — and using their structure to support interpretation

How an Ensemble Works



Interpretability idea

- Multiple simple models can collectively capture complex patterns.
- Ensemble behavior can be examined using feature contributions, partial dependence, or model structure.

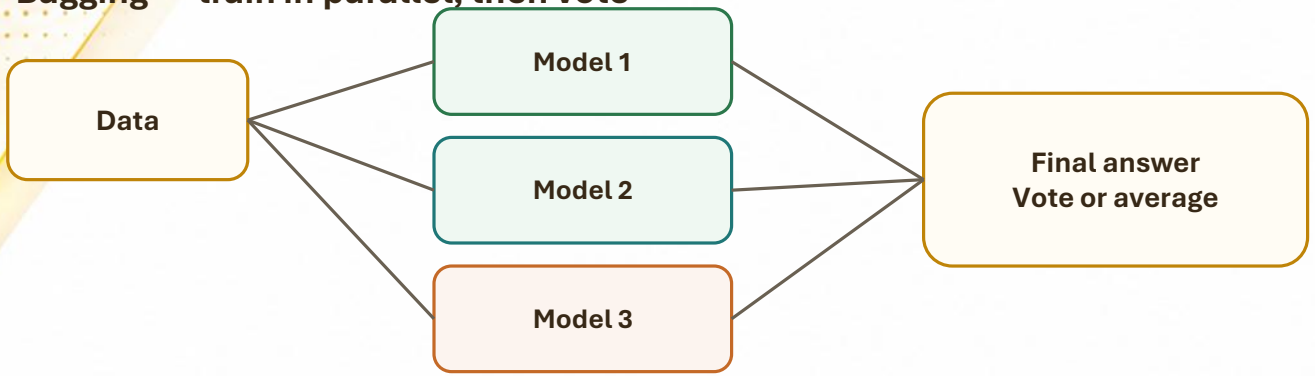
Common Ensemble Approaches

1. **Bagging / Random Forest**
   Many trees → aggregate predictions
2. **Boosting**
   Models learn sequentially from errors
3. **Voting / Stacking**
   Combine different model predictions
4. **Interpretable Ensembles**
   Use constrained or additive components for transparent behavior

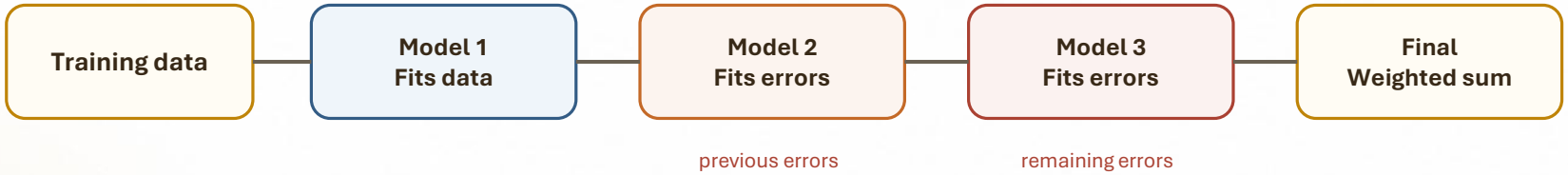
Note: Random Forest/XGBoost are not inherently simple.

Bagging vs. Boosting vs. Stacking

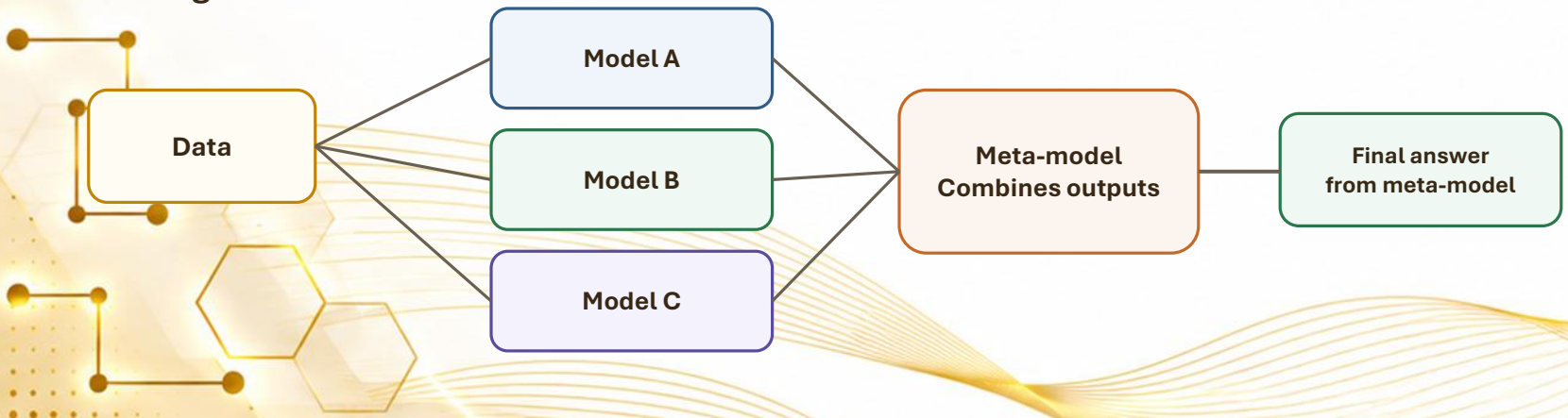
Bagging — train in parallel, then vote



Boosting — train in sequence; each model learns from previous errors



Stacking — diverse models feed a meta-learner



At a glance

BAGGING
Parallel

BOOSTING
Sequential

STACKING
Meta-model

All three combine multiple models
— but the models are combined in different wa

Bagging with Random Forest – Example

1. Bootstrap Sampling

Create multiple random samples (with replacement) from the original data.

2. Train Multiple Decision Trees

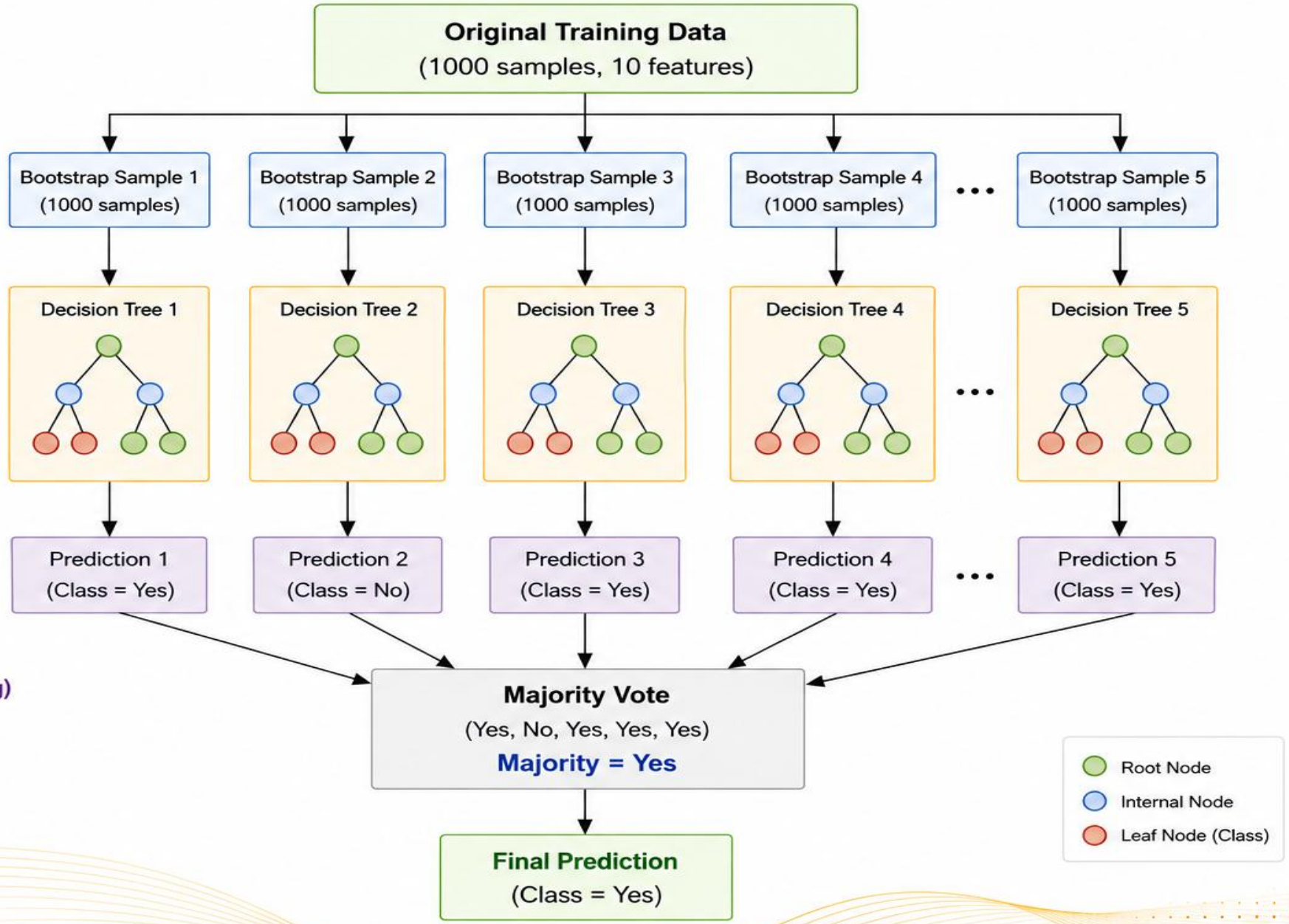
A decision tree is trained on each bootstrap sample.

3. Make Predictions

Each tree makes a prediction for a new input.

4. Aggregate (Majority Voting)

The final prediction is the majority vote of all trees.



Boosting – Example (AdaBoost with Decision Trees)

Example Problem: Predict whether a customer will **Buy (Yes)** or **Not Buy (No)** a product based on two features: Age and Income.

1. Initialize Weights

Assign equal weight to all training samples.

$$w_i = \frac{1}{N} = \frac{1}{10} = 0.1 \quad \text{for all } i$$

2. Train Weak Learner (Decision Tree)

Train a decision tree using the current sample weights.

3. Evaluate & Update Weights

Compute error, then increase weights of misclassified samples and decrease weights of correctly classified samples.

4. Combine Learners

Final prediction is a weighted majority vote of all weak learners.

Original Training Data (10 samples)

ID	1	2	3	4	5	6	7	8	9	10
Age	22	25	28	35	35	45	48	52	55	60
Income (₹K)	30	40	25	50	35	60	45	55	50	70
Actual (Buy?)	Yes	Yes	No	Yes	No	Yes	No	Yes	No	Yes

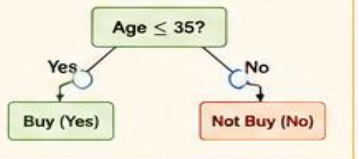
Round 1

Sample Weights (all equal)

0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1

Train Decision Tree $h_1(x)$

(Example tree based on Age)



Predictions vs Actual

Y	Y	N	Y	Y	Y	N	N	Y	

Error $\epsilon_1 = 2/10 = 0.20$

Update Weights (higher for misclassified)

0.083	0.083	0.167	0.083	0.167	0.083	0.083	0.167	0.167	0.083

Classifier Weight

$$\alpha_1 = \frac{1}{2} \ln \frac{1-\epsilon_1}{\epsilon_1} = 0.693$$

Round 2

New Sample Weights

0.083	0.083	0.167	0.083	0.167	0.083	0.083	0.167	0.167	0.083

Train Decision Tree $h_2(x)$

(Example tree based on Income)



Predictions vs Actual

Y	Y	N	Y	N	N	Y	N	Y	

Error $\epsilon_2 = 2/10 = 0.20$

Update Weights (higher for misclassified)

0.069	0.069	0.139	0.069	0.278	0.069	0.069	0.139	0.139	0.069

Classifier Weight

$$\alpha_2 = \frac{1}{2} \ln \frac{1-\epsilon_1}{\epsilon_1} = 0.693$$

Round 3

New Sample Weights

0.069	0.069	0.139	0.069	0.278	0.069	0.069	0.069	0.139	0.278	0.069

Train Decision Tree $h_3(x)$

(Example tree based on Age & Income)



Predictions vs Actual

Y	Y	N	Y	Y	N	N	Y	N	Y

Error $\epsilon_3 = 1/10 = 0.10$

Update Weights (higher for misclassified)

0.061	0.061	0.123	0.061	0.491	0.061	0.061	0.123	0.491	0.061

Classifier Weight

$$\alpha_3 = \frac{1}{2} \ln \frac{1-\epsilon_2}{\epsilon_3} = 1.099$$

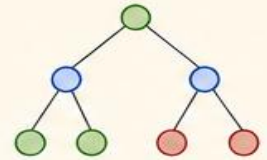
Round K (Final)

New Sample Weights

0.061	0.061	0.123	0.061	0.491	0.061	0.061	0.123	0.491	0.061

Train Decision Tree $h_K(x)$

(Final weak learner)



Predictions vs Actual

Y	Y	N	Y	N	N	Y	N	Y	

Error $\epsilon_K = 0.10$ (example)

Update Weights (final)

... (would update again if more rounds)

Classifier Weight

$$\alpha_K = 1.099 \text{ (example)}$$

Weighted Majority Vote

$$F(x) = sign\left(\sum_{t=1}^{K} \alpha_t \; h_t(x)\right), \quad \text{where} \quad h_t(x) \in \{-1, +1\} \quad (Not\ Buy = -1,\ Buy = +1)$$

Final Prediction for a New Customer

(Age = 38, Income = 42K)

$$F(x) = +1 \rightarrow \text{Buy (Yes)}$$

Stacking (Stacked Generalization) – Example

Combining different models using a meta-learner

1. Base Models

Train different types of models on the training data.

2. Generate Base-Model Predictions

Each base model makes predictions for the training data (usually using k-fold cross-validation to avoid overfitting).

3. Train Meta-Learner

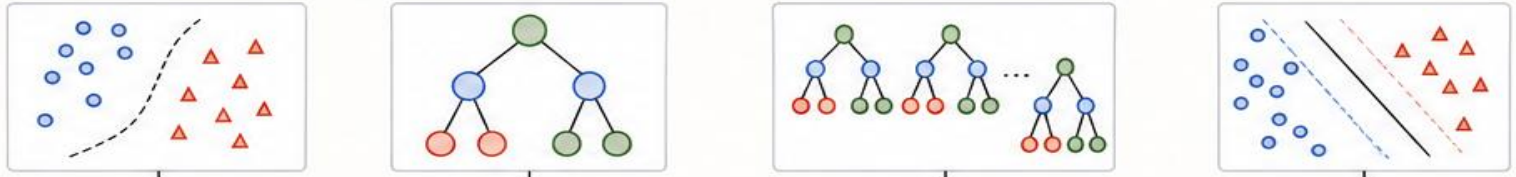
Train a meta-learner using the base-model predictions as input features and the true target as output.

4. Final Prediction

The meta-learner combines the base-model predictions and produces the final output.

Training Data
(Features X, Target y)

Model 1	Model 2	Model 3	Model 4
(Logistic Regression)	(Decision Tree)	(Random Forest)	(SVM)

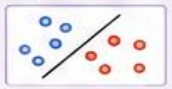


Prediction 1	Prediction 2	Prediction 3	Prediction 4
(Probability / Class)	(Probability / Class)	(Probability / Class)	(Probability / Class)

New Training Data for Meta-Learner (Meta-Features)
(Each row = one original sample, each column = base model's prediction)

Sample ID	Pred 1	Pred 2	Pred 3	Pred 4	...	True Target (y)
1	0.62	0.70	0.65	0.60	...	1
2	0.20	0.10	0.15	0.20	...	0
3	0.75	0.80	0.78	0.70	...	1
...	...	...	...	...	...	...
n	0.30	0.20	0.25	0.28	...	0

Meta-Learner
(e.g., Logistic Regression)



Final Prediction for x_{new}
(Probability / Class)

New Test Sample
(x_{new})

- Model 1 prediction
- Model 2 prediction
- Model 3 prediction
- Model 4 prediction

Meta-Features for x_{new}

p1	p2	p3	p4

Ensembled Based Tools for interpretability

- Boosted Rulesets
- Skope-Rules
- RuleFit
- Explainable Boosting Machines (EBM)
- Iterative Random Forest (iRF)

Boosted Rulesets – Example

Building a strong model by adding simple IF–THEN rules sequentially

1 Start with data

All training examples have equal weight.

ID	Income (₹)	Credit Score	Loan	Default (y)
1	20,000	580	Yes	1
2	35,000	670	No	0
3	55,000	720	No	0
4	40,000	610	Yes	1
5	28,000	590	Yes	1
...	...	...	...	...
n	...	...	...	...

(1 = Default, 0 = No Default)

2 Learn Rule 1

Find the rule that best separates the weighted errors (highest improvement).

Rule 1

IF Income < ₹30,000

THEN Predict: Default (1)

Covers examples: {1, 5, ...}

Weighted error $(\varepsilon_1) = 0.32$

Update weights: Increase weights of misclassified examples, decrease others.

3 Learn Rule 2

Learn next best rule on the updated weights.

Rule 2

IF Credit Score < 620

AND Loan = Yes

THEN Predict: Default (1)

Covers new examples: {4, ...}

Weighted error $(\varepsilon_2) = 0.35$

Update weights: Again increase weights of wrongly classified examples.

M Learn Rule M

Continue adding rules until performance stops improving or max rules reached.

Rule M

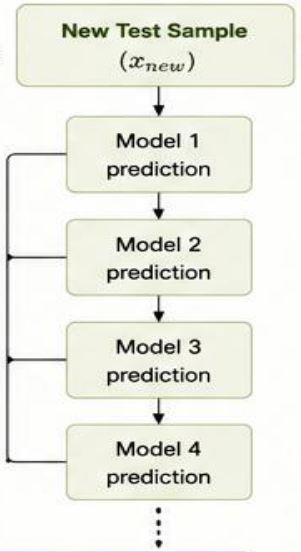
IF Credit Score ≥ 700

AND Income ≥ ₹40,000

THEN Predict: No Default (0)

Weighted error $(\varepsilon_M) = 0.18$

Update weights: Final weight update after Rule M.



4 Final Boosted Ruleset (Additive Model)

Each rule has a weight (α_m) based on its accuracy.

Rule No.	IF (Condition)	THEN (Prediction)	Weight (α_m)
1	Income < ₹30,000	Default (1)	0.84
2	Credit Score < 620 AND Loan = Yes	Default (1)	0.67
3	Income < ₹25,000 AND Credit Score < 680	Default (1)	0.51
...	...	...	...
M	Credit Score ≥ 700 AND Income ≥ ₹40,000	No Default (0)	0.48

Model prediction (for any case x):

$$F(x) = \sum_{m=1}^{M} \alpha_m \cdot h_m(x)$$

where $h_m(x) = 1$ if rule m fires for x, else 0

Example

Total score $F(x) = 1.25$ (positive)

⇒ **Predict: Default (1)**

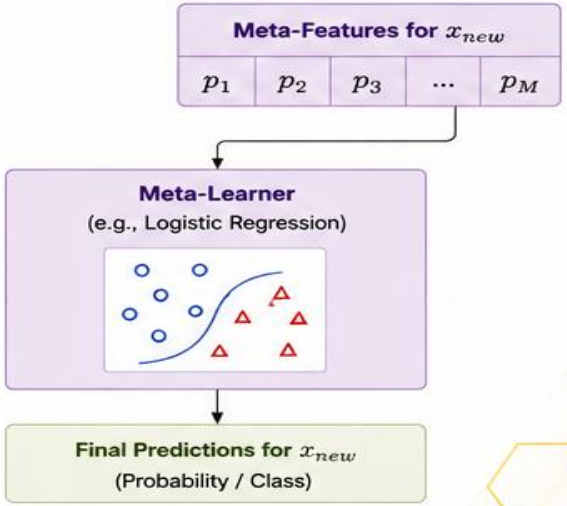
5 How a New Case is Predicted

New Customer
Income = ₹27,000
Credit Score = 610
Loan = Yes

Check Rules in Order
(Top to Bottom)

Rule 1: Income < ₹30,000 → Fires (1) ✓
Rule 2: Credit Score < 620 AND Loan = Yes → Fires (1) ✓
Rule 3: Income < ₹25,000 AND Credit Score < 680 → No ✗
...
Rule M: Credit Score ≥ 700 AND Income ≥ ₹40,000 → No ✗

Score = 0.84 + 0.67 = 1.51 (> 0)
Final Prediction = Default (1)



Explainable properties of boosted rulesets

Properties	Values	Properties	Values
Local or global	Global and local	Feature interactions captured	Yes
Linear or non-linear	Non-linear (can be axis parallel)	Model complexity	Medium to high
Monotonic or non-monotonic	Non-monotonic		

Skope-Rules – Example

Extracting high-quality, human-readable rules from tree ensembles

1 Train a Tree Ensemble

Train a Random Forest on the training data.

Training Data (Loan Default)

ID	Income (₹)	Credit Score	Loan Amount (₹)	Default (1/0)
1	20,000	580	1,00,000	1
2	35,000	670	1,50,000	0
3	55,000	720	2,00,000	0
4	40,000	610	1,20,000	1
5	28,000	590	80,000	1
⋮	...	⋮	⋮	⋮
n	...	⋮	⋮	...

(1 = Default, 0 = No Default)

2 Generate Rule Candidates

Collect many IF–THEN rule candidates from all trees.

Examples of Rule Candidates

- IF Income < ₹30,000 THEN Default (1)
- IF Credit Score < 620 THEN Default (1)
- IF Loan Amount > ₹1,50,000 THEN Default (1)
- IF Income ≥ ₹25,000 AND Credit Score < 680 THEN Default (1)

...

3 Evaluate Candidates

Score each rule by quality measures (precision, recall, F1, coverage, etc.).

Candidate Evaluation

Rule ID	Precision	Recall	F1-Score	Coverage
R_1	0.75	0.60	0.67	0.30
R_2	0.78	0.55	0.64	0.25
R_3	0.72	0.40	0.52	0.18
R_4	0.86	0.50	0.63	0.20
...	⋮	⋮	⋮	⋮

Rules are scored using weighted combination of quality metrics.

4 Select Non-overlapping High-quality Rules

Choose a small set of rules that are accurate and minimally overlapping.

Selected Skope-Rules

S_1: IF Income < ₹30,000 → Default (1)

S_2: IF Credit Score < 620 AND Loan Amount > ₹1,00,000 → Default (1)

S_3: IF Credit Score ≥ 700 AND Income ≥ ≥ ₹40,000 → No Default (0)

Default Rule: → No Default (0) (if none of the above match)

5 Final Interpretable Model

A compact, human-readable set of rules.

Skope-Rules Model

IF Income < ₹30,000
THEN Default (1)

ELSE IF Credit Score < 620 AND Loan Amount > ₹1,00,000
THEN Default (1)

ELSE IF Credit Score ≥ 700 AND Income ≥ ₹40,000
THEN No Default (0)

ELSE
THEN No Default (0)

6 How a New Case is Predicted

New Customer

Income = ₹27,000
Credit Score = 610
Loan ≥ Yes

→ **Check Rules in Order (Top to Bottom)** →

Rule	IF Condition	THEN	Precision	Recall	Coverage
S_1	Age ≥ 50 AND Debt-to-Income > 40%	Default (1)	0.82	0.58	0.22
S_2	Employment = Unstable AND Credit Score < 650	Default (1)	0.76	0.52	0.19
S_3	Credit Score ≥ 750 AND Loan Amount < ₹1,00,000	No Default (0)	0.88	0.60	0.24
Default Rule	(None of the above)	No Default (0)	–	–	–

Score = 0.84 + 0.67 = 1.51 (> 0)
Final Prediction = Default (1)

7 Meta-Learner

A meta-learner (e.g., Logistic Regression) combines the rule predictions.

Meta-Features for x_{new}

p_1 p_2 p_3 p_4 ... p_M

↓

Meta-Learner
(e.g., Logistic Regression)



↓

Final Prediction for x_{new}
(Probability / Class)

8 Explainable Properties of Skope-Rules

Properties	Values
Local or global	Global and local
Linear or non-linear	Non-linear (can be axis parallel)
Monotonic or non-monotonic	Non-monotonic
Feature interactions captured	Yes
Model complexity	Medium to high

Interpretable • Accurate • Human-Readable

RuleFit – Example

Combining rules from trees with a linear model

1 Train Tree Ensemble

Train a Random Forest (bagging) on the training data.

Training Data

ID	Income (₹)	Credit Score	Loan (Yes/No)	Default (1/0)
1	20,000	580	Yes	1
2	35,000	670	No	0
3	55,000	720	No	0
4	40,000	610	Yes	1
5	28,000	590	Yes	1
...	...	...	...	...

(1 = Default, 0 = No Default)

2 Extract Rules from Trees

Collect useful IF–THEN rules from all trees in the ensemble.

Example Rules Extracted

R_1: IF Income < ₹30,000 THEN Default (1) → 1

R_2: IF Credit Score < 620 AND Loan = Yes → 1

R_3: IF Income < ₹25,000 AND Credit Score < 680 → 1

⋮

R_P: IF Credit Score ≥ 700 AND Income ≥ ₹40,000 → 0

Each rule R_i becomes a binary feature (1 if condition true, else 0).

3 Build Rule + Linear Features

Combine rule features with original linear features and fit a sparse linear model.

Design Matrix for Linear Model

Income	Credit Score	Loan (Yes)	R_1	R_2	R_3	...	R_P	y
20000	580	1	1	1	1	...	0	1
35000	670	0	0	0	0	...	0	0
55000	720	0	0	0	0	...	0	0
40000	610	1	0	1	0	...	0	1
28000	590	1	1	1	1	...	0	1
...	...	...	...	...	...	...	...	...

Fit Sparse Linear Model ($\mathcal{L}_1$ regularization)

$$\hat{y} = \beta_0 + \sum_{k=1}^{K} \beta_k x_k + \sum_{j=1}^{P} \gamma_j R_j$$

(x_k = original features, R_j = rule features)

4 Final RuleFit Model

Keep only rules and features with non-zero coefficients.

Learned RuleFit Model

Feature / Rule	Coefficient	Importance
Intercept (β_0)	−2.35	—
Income (β_1)	0.000045	High
Credit Score (β_2)	0.0062	High
Loan = Yes (β_3)	0.48	Medium
R_1: Income < ₹30,000	1.25	High
R_2: Credit < 620 & Loan=Yes	0.95	High
R_3: Income < ₹25,000 & Credit < 680	0.62	Medium
...	...	...
R_P: Credit ≥ 700 & Income ≥ 40k	−1.10	High

Prediction for any case x:

$$\hat{y} = \beta_0 + \sum \beta_k x_k + \sum \gamma_j R_j$$

5 How a New Case is Predicted

New Customer

Income = ₹27,000

Credit Score = 610

Loan = Yes

Evaluate Rules

R_1: Income < ₹30,000 → 1 (True)

R_2: Credit < 620 AND Loan=Yes → 1 (True)

R_3: Income < ₹25,000 AND Credit < 680 → 0 (False)

... ...

R_P: Credit ≥ 700 AND Income ≥ ₹40,000 → 0 (False)

Compute Score

$$\text{Score} = \beta_0 + \beta_1(27{,}000) + \beta_2(610) + \beta_3(1) + \gamma_1(1) + \gamma_2(1) + \gamma_3(0) + \ldots + \gamma_P(0)$$

$$= -2.35 + 0.000045 \times 27000 + 0.0062 \times 610 + 0.48 \times 1 + 1.25 \times 1 + 0.95 \times 1 + \ldots$$

$$= 1.23 \quad \text{(positive)}$$

⇒ **Predict: Default (1)**

Explainable Properties of RuleFit

Properties	Values	Properties	Values
Local or global	Global and local	Feature interactions captured	Yes
Linear or non-linear	Non-linear (can be axis parallel)	Model complexity	Medium to high
Monotonic or non-monotonic	Non-monotonic		

Explainable Boosting Machine (EBM) – Example

EBM is an interpretable, additive model that shows how each feature contributes to the prediction.

1 Data and Task

Predict house price (in lakh ₹) using important features.

Training Data (Example)

Area (sq.ft.)	Bedrooms (no.)	Age (years)	Location (score)	Price (lakh ₹)
1200	2	10	6	45
1500	3	5	7	60
1800	3	8	8	75
2000	4	3	9	90
2500	4	6	9	110
...	...	...	...	...

Goal:
Predict Price (continuous) using an interpretable model.

2 EBM Model (Additive Form)

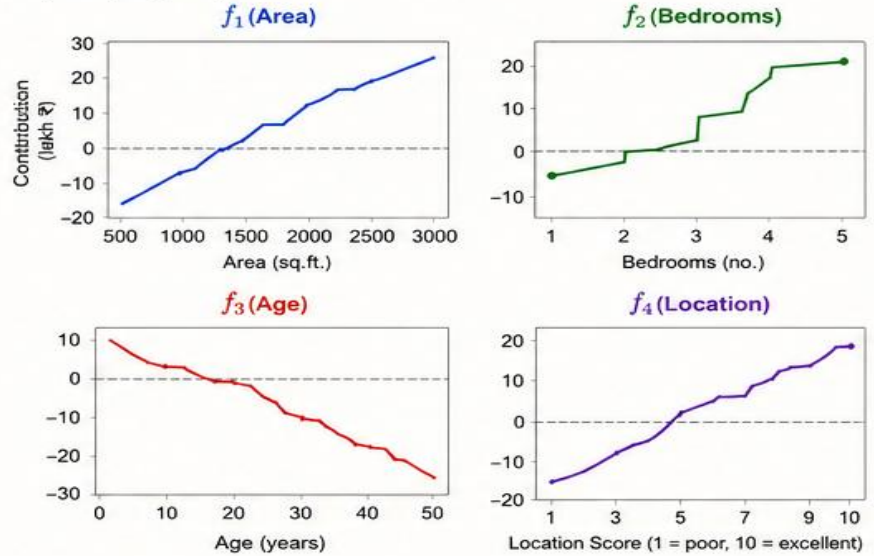
EBM models the prediction as the sum of individual feature effects + intercept.

$$\hat{y} = \beta_0 + f_1(\text{Area}) + f_2(\text{Bedrooms}) + f_3(\text{Age}) + f_4(\text{Location})$$

- β_0 = global intercept (baseline prediction)
- $f_j(x_j)$ = learned shape function for feature j
- Each f_j shows how that feature changes the prediction when the others are held at their average values.

3 Learned Feature Functions

Each feature gets its own shape function. (Example plots)



4 Global Intercept (Baseline)

The average prediction when all features are at their reference (average) values.

Intercept (β_0) = 50.0 lakh ₹

Feature Functions Summary

Feature	Effect on Price
Area	Larger area → increases price
Bedrooms	More bedrooms → increases price
Age	Higher age → decreases price
Location	Better location → increases price

5 Explain a New Prediction

New house:

- Area = 1800 sq.ft.
- Bedrooms = 3
- Age = 8 years
- Location Score = 8

Read Contributions from Plots

Feature	Value	Contribution $f_j(x_j)$ (lakh ₹)
Area	1800	+12.0
Bedrooms	3	+8.0
Age	8	−4.0
Location	8	+12.0
Intercept (β_0)	–	**+50.0**

Add Contributions (Additive)

$$\hat{y} = \beta_0 + f_1(\text{Area}) + f_2(\text{Bedrooms}) + f_3(\text{Age}) + f_4(\text{Location})$$
$$= 50.0 + 12.0 + 8.0 - 4.0 + 12.0$$
$$= \mathbf{78.0 \text{ lakh } ₹}$$

Predicted House Price = 78.0 lakh ₹

Explainable Properties of EBM

Properties	Values
Local or global	Global and local
Linear or non-linear	Non-linear
Monotonic or non-monotonic	Monotonic
Feature interactions captured	Yes
Model complexity	High

Iterative Random Forests (iRF) – Example

Discovering important feature interactions learned by a Random Forest

1 Train a Random Forest (Model)

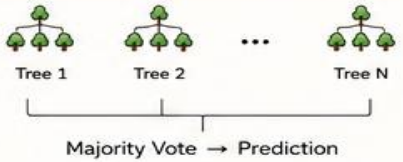
Train a Random Forest on the data.

Example: Disease Prediction

Target: Disease (1 = Yes, 0 = No)

Features:

- Age (years)
- Blood Pressure (BP)
- Cholesterol (Chol)
- Diabetes (Yes/No)
- Smoking (Yes/No)
- Family History (Yes/No)



2 Generate Feature Combinations

Create combinations of features (1-way, 2-way, 3-way, …).

Examples of Combinations

1-way	Age
2-way	Age & BP
2-way	Chol & Diabetes
3-way	Age, BP & Diabetes
3-way	BP, Chol & Smoking
…	…
k-way	… …

iRF iteratively examines higher-order interactions.

3 Evaluate Each Combination

For each combination, create groups (bins) based on the features and evaluate how well the RF predicts within those groups.

Example: Interaction = Age & BP

Age Group (years)	BP Group (mmHg)		
	<120	120–139	≥140
<40	0.05	0.12	0.35
40–60	0.08	0.18	0.55
>60	0.18	0.38	0.70

Value = Disease rate (1 = Yes) predicted by RF in each cell.

Good interactions create groups with very different predictions (low impurity).

4 Rank Interactions

Score all combinations using a metric (e.g., variance of predictions, mutual information, or ANOVA F-statistic).

Top Interactions (Higher Score = More Important)

Rank	Interaction	Size	Score
1	Age, BP & Diabetes	3-way	0.246
2	BP & Cholesterol	2-way	0.183
3	Age & Diabetes	2-way	0.152
4	Smoking & Cholesterol	2-way	0.101
5	Age & BP	2-way	0.097
…	…	…	…

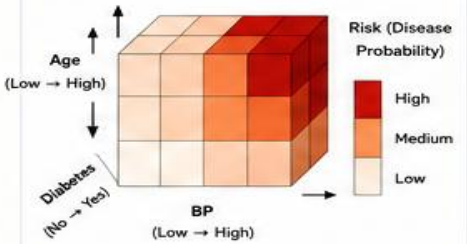
Select the most important interactions for interpretation.

5 Visualize & Interpret

Present the top interactions as simple rules or heatmaps.

Example: Top Interaction Age, BP & Diabetes

If Age > 50
AND BP > 140
AND Diabetes = Yes
THEN High Risk of Disease



Example Walkthrough

New Patient

Age = 62 years
BP = 150 mmHg
Cholesterol = High
Diabetes = Yes
Smoking = No
Family History = Yes

Top Interaction Found by iRF (Age, BP & Diabetes)

Heatmap of Disease Risk (from RF predictions)

Age Group (years)	BP Group (mmHg)					
	<120		120–139		≥140	
	Diabetes No	Yes	Diabetes No	Yes	Diabetes No	Yes
<40	0.03	0.07	0.08	0.15	0.18	0.30
40–60	0.06	0.12	0.15	0.25	0.30	0.55
>60	0.12	0.22	0.28	0.45	0.40	0.72

Risk is highest when Age, BP and Diabetes are high.

Locate Patient in the Interaction

Age = 62 → >60 row
BP = 150 → ≥140 column
Diabetes = Yes → 'Yes' column

Predicted Disease Risk

(From RF in this cell)

0.72 (High Risk)

Interpretation

The model indicates high risk because all three factors are in the high-risk region together.

Simple Rule from iRF

IF Age > 50
AND BP > 140
AND Diabetes = Yes
THEN High Risk of Disease

Explainable Properties of iRF

Properties	Values
Local or global	Global and local
Linear or non-linear	Non-linear
Monotonic or non-monotonic	Non-monotonic
Feature interactions captured	Yes
Model complexity	Medium to high